Empirical Evaluation of Temporal-Structural Leakage in GNN-Based Fraud Detection

Gilbert Woasieworm Adjei
MPhil Data Science
22425084
gwadjei@st.ug.edu.gh

Fred Achi
MSc Data Science
22424751
achifred123@gmail.com

Philip Buetey Adiboye
MSc Data Science
22427838
pbadiboye@st.ug.edu.gh

Abstract

Fraud detection systems built on blockchain transaction graphs are often evaluated with a random train and test split. This is a problem, because these graphs change over time, and models like Graph Convolutional Networks (GCN) and GraphSAGE predict by aggregating information from a node's neighbours. A random split lets edges from future time steps leak into training, and lets test nodes draw on information from nodes not yet labelled at the time being predicted. We call this temporal-structural leakage and set out to measure it directly. We trained six models, Logistic Regression, XGBoost, an MLP, GCN, GraphSAGE, and an EvolveGCN built from scratch, on the Elliptic Bitcoin dataset (203,769 nodes, 234,355 directed edges, 49 biweekly time steps). Each model was evaluated once under a naive random 70/10/20 split, and once under a strict chronological split (train on steps 1 to 34, validate on 35 to 39, test on 40 to 49) with causal edge masking enforced throughout. Averaged over three seeds, every model's F1 on the illicit class dropped once the leaky shortcut was removed, from 21.7 points for GCN up to 55.0 points for MLP. Even XGBoost, which never touches a graph edge, lost 29.0 points from feature-level leakage alone. A feature-importance breakdown showed local transaction features drove more of this leakage than the neighbourhood-aggregate features we expected to be the main culprit, and an edge-masking ablation found stripping the graph to zero edges only closed the gap slightly. This points to leakage that lives substantially in the feature columns themselves, not only in the graph structure. We also found our EvolveGCN implementation stayed unstable across seeds even after gradient clipping, showing the honest evaluation condition is harder not just in the numbers it produces but in how reliably the model trains at all.

1. Introduction

Financial institutions are increasingly turning to graph neural networks to flag suspicious transactions, on the reasonable assumption that fraud tends to spread through networks of connected accounts. Published results on benchmarks like the Elliptic dataset often report F1 scores above 0.90 for these systems. The trouble is that most of these numbers come from a random train/test split, which quietly breaks the temporal ordering baked into the underlying transaction graph. A model evaluated this way can pull in information from neighbours that occur later in time than the transaction it is trying to classify, and it can be trained on edges that would not have existed yet at the point of real deployment. We refer to this as temporal-structural leakage, to separate it from the more familiar kind of leakage that comes from duplicated rows or shared preprocessing statistics. This kind operates through the graph's edges.

The gap this creates between a reported benchmark and real deployment performance has real consequences. A system that looks like it catches 91% of fraud but actually catches closer to 73% will miss a lot more illicit activity than an institution has been told to expect. Our goal in this project was to measure that gap precisely, one architecture at a time, and to figure out whether the leakage mostly comes from message passing over future edges or whether it is baked into the node features themselves. This second possibility matters because a large share of the Elliptic dataset's 165 features are themselves neighbourhood aggregates, computed with knowledge of the full graph.

We trained six models under two evaluation pipelines: three baselines with no access to graph structure (Logistic Regression, XGBoost, an MLP), and three graph-aware models (GCN, GraphSAGE, and a temporally-aware EvolveGCN we implemented ourselves). Averaged over three seeds, every architecture lost a substantial amount of F1 once we removed the leaky shortcut. The drop was not limited to the graph-based models either. XGBoost, which never sees an edge, still lost 29 points. And when we re-

moved graph edges entirely to test whether the leakage was purely structural, the gap barely moved, suggesting a meaningful chunk of it is encoded directly in the feature columns.

2. Related Work

Our graph models build on a few established papers. Kipf and Welling [3] introduce the Graph Convolutional Network under a transductive setup that assumes the entire graph, test nodes and their neighbours included, is available at training time. That assumption is exactly what makes a random split unsafe for a graph that evolves over time, since it lets future information reach past predictions. Hamilton *et al*. [1] propose GraphSAGE, whose inductive, sampling-based approach to aggregation is meant to generalise to nodes it has not seen before. Going into this project we expected that inductive design to handle temporal boundaries somewhat better than GCN's transductive one. Pareja *et al*. [5] introduce EvolveGCN, which evolves a GNN's weight matrices over time using a recurrent unit instead of keeping them fixed, making it the natural temporally-aware model to include here. No stable, ready-made implementation of it exists in PyTorch Geometric, so we wrote the recurrent weight-evolution layer ourselves.

Weber *et al*. [6] introduce the Elliptic Bitcoin dataset and, notably, already evaluate it under a chronological 70:30 split rather than a random one, training on the first 34 time steps and testing on the remaining 15. Under that protocol they report an illicit F1 of 0.481 for Logistic Regression, 0.788 for Random Forest, 0.653 for an MLP, 0.705 for their GCN variant, and 0.720 for EvolveGCN. Our own temporal-split numbers come out lower across every comparable model (Logistic Regression 0.255, GCN 0.430, EvolveGCN 0.124), despite following a similar chronological protocol. We think this gap is informative rather than a discrepancy to explain away. Our causal edge masking blocks any message passing across a future time boundary at both training and evaluation time, a constraint the original paper does not describe enforcing in this much detail, so our lower numbers may reflect a stricter, more honest protocol rather than a weaker implementation. What this comparison does make clear is that the leaky random-split numbers commonly quoted for this dataset in later work, several papers report GCN accuracy above 98%, diverge sharply from the dataset's own original, already-temporal baseline. That suggests the drift toward random splits happened gradually across the literature that followed Weber *et al*., rather than being present from the start.

Kaufman *et al*. [2] provide the general taxonomy of data leakage that our terminology draws from. What we are looking at fits inside their broader category of leakage introduced during data collection, specialised here to a graph-structured, temporally-ordered setting.

A contemporary and closely related study, Maganti [4], reaches a similar conclusion on the same dataset independently. Using a seed-matched inductive versus transductive comparison, that work finds Random Forest on raw features beats every GNN tested once test-period adjacency is properly withheld, and reports a 39.5-point paired F1 gap attributable purely to training-time exposure to future edges, closely matching the direction and rough scale of our own leakage delta. Where our project differs is in scope and in mechanism. We cover a wider set of architectures, including a from-scratch EvolveGCN and three non-graph baselines, and we go a step further in asking where the leakage actually lives, breaking it down into a structural component tested through progressive edge masking and a feature-level component tested through a local versus neighbourhood importance comparison. Our finding that removing all graph edges only narrows the gap slightly complements that paper's edge-shuffle result and points toward the same underlying conclusion from a different angle, that a meaningful share of this dataset's apparent graph advantage is really an artefact of temporal leakage rather than genuine structural signal.

Where this project differs from the papers above, taken as a whole, is that we are not proposing a new architecture. We are treating the evaluation protocol itself as the thing worth studying, running the same six models under two protocols and then trying to explain the resulting gap through an edge-masking ablation and a comparison of local versus neighbourhood feature importance.

3. Method

3.1. Two evaluation pipelines

Pipeline A, the random pipeline, shuffles every labelled node and splits it 70/10/20 into train, validation, and test, without regard to time step. Pipeline B, the temporal pipeline, splits strictly by time. Nodes from steps 1 through 34 form the training set, steps 35 through 39 form validation, and steps 40 through 49 form the held-out test set. Both pipelines run on the same underlying graph of 165 features and 234,355 edges. Only how the labelled nodes get divided into train, validation, and test changes between them.

For the three non-graph baselines, that split is the only difference between pipelines. For the graph-based models, Pipeline B adds one more constraint: causal edge masking. At every training or evaluation step restricted to time t or earlier, the model can only pass messages across edges whose endpoints both occur at time t or earlier. This is handled by a `causal_edge_index` function that filters the full edge index before every forward pass, and it is what stops a node from time step 40 from ever picking up information through a node at time step 45, even indirectly through a chain of edges.

3.2. Models

Logistic Regression and XGBoost work only from the 165 node features, with no view of the graph at all, so they give us a lower bound on how much leakage the features alone can carry. The MLP, two hidden layers of 128 units with batch normalisation and dropout of 0.3, also ignores edges. GCN and GraphSAGE, both two convolutional layers with a hidden size of 128 and dropout of 0.5, use standard PyTorch Geometric implementations. These values were selected based on common practice for node classification on graphs of this scale, and kept fixed across GCN and GraphSAGE so that the comparison between the two architectures reflects structural differences rather than tuning advantages. EvolveGCN uses a smaller hidden size of 64 rather than 128, a practical necessity given the added computational cost of its per-row recurrent weight update, not a tuning choice. EvolveGCN was the one model that needed a full custom implementation. Each layer keeps a weight matrix that gets updated recurrently by a GRU cell, conditioned on a summary of the current snapshot's node features, before that weight matrix is applied as a graph convolution with degree normalisation. Training this recurrent, multi-snapshot model turned out to be far less stable than the two static GNNs. We ended up needing gradient-norm clipping, capped at a max norm of 1.0, just to keep the loss from diverging, and even with clipping in place, training still came out sensitive to which random seed we used, discussed further in Section 4.5.

3.3. Alternatives considered

We looked at using an existing temporal-GNN library for EvolveGCN instead of writing our own. We decided against it so that we could control the causal-masking mechanism directly and be certain, in code we wrote and tested ourselves, that no message ever crosses from a future time step into the past, rather than trusting that guarantee to an external library we had not fully audited.

4. Experiments and Results

4.1. Data

We used the Elliptic Bitcoin Transaction dataset [6], downloaded through the Kaggle API. It contains 203,769 transaction nodes and 234,355 directed edges spanning 49 biweekly time steps, with 165 features per node, 94 describing local transaction attributes and 71 describing neighbourhood-level aggregates. Of the 203,769 nodes, 4,545 (2.2%) are labelled illicit, 42,019 are labelled licit, and the remaining 157,205 are unlabelled and excluded from both splits. We treat the edge list as undirected for message passing by storing both directions, and we fit feature scaling only on each split's training partition, to avoid a second, less obvious form of leakage through the scaler's own statistics. Given how imbalanced the labelled classes are, illicit nodes make up just 2.2% of them, we report F1, precision, and recall on the illicit class throughout, and we deliberately leave accuracy out as a headline metric.

Model	F1 (random)	F1 (temp.)	Δ
MLP	0.7924 ± 0.0066	0.2428 ± 0.2030	0.5496
EvolveGCN	0.6258 ± 0.0489	0.1239 ± 0.1317	0.5019
LR	0.6034 ± 0.0168	0.2550 ± 0.0000	0.3483
GraphSAGE	0.7786 ± 0.0154	0.4691 ± 0.0184	0.3095
XGBoost	0.9590 ± 0.0026	0.6688 ± 0.0039	0.2902
GCN	0.6472 ± 0.0169	0.4302 ± 0.0231	0.2170

Table 1. F1 (mean $\pm$ std over 3 seeds) under random versus temporal evaluation, and the resulting leakage delta.

Model	Split	Precision	Recall
LR	Random	0.4447 ± 0.0186	0.9388 ± 0.0039
LR	Temp.	0.1505 ± 0.0000	0.8365 ± 0.0000
XGBoost	Random	0.9687 ± 0.0082	0.9495 ± 0.0069
XGBoost	Temp.	0.7520 ± 0.0095	0.6022 ± 0.0016
MLP	Random	0.6862 ± 0.0154	0.9379 ± 0.0114
MLP	Temp.	0.5051 ± 0.4290	0.3994 ± 0.3418
GCN	Random	0.5037 ± 0.0200	0.9056 ± 0.0025
GCN	Temp.	0.4387 ± 0.0457	0.4235 ± 0.0051
GraphSAGE	Random	0.6682 ± 0.0202	0.9330 ± 0.0101
GraphSAGE	Temp.	0.4197 ± 0.0330	0.5335 ± 0.0126
EvolveGCN	Random	0.4805 ± 0.0577	0.9036 ± 0.0108
EvolveGCN	Temp.	0.0833 ± 0.0838	0.2552 ± 0.3114

Table 2. Precision and recall (mean $\pm$ std over 3 seeds) under both splits.

4.2. Main result: the leakage delta

Table 1 reports F1 on the illicit class under both pipelines, averaged over three seeds (42, 123, 456), along with the leakage delta, defined as F1 under the random split minus F1 under the temporal split.

Every single model loses a substantial amount of F1 once evaluated honestly. That drop is not something graph access alone explains. XGBoost, which never sees an edge, still loses 29.0 points, or 30.3% of its random-split score, purely because its training features summarise a state of the graph that would not exist yet at real deployment time. MLP shows the biggest relative drop at 69.4%, and its temporal-split standard deviation of 0.203 is unusually wide, something we come back to in Section 4.5. Table 2 breaks the same results down into precision and recall.

The precision and recall numbers tell their own story. Under the random split, every model reaches very high recall, between 0.90 and 0.95, at the cost of only moderate precision, which looks a lot like a model that has learned structural shortcuts rather than genuine fraud signal. Under the temporal split, recall falls much further than precision for the graph-based models. GraphSAGE's recall, for example, drops from 0.933 down to 0.534. In practical terms, that means a model that looks like it catches most fraud under the leaky protocol would actually miss roughly half of

it once deployed honestly.

4.3. F1 across time

Table 1 averages over the full test period, which hides how uneven performance is across time. Figure 1 breaks that average apart.

Two patterns stand out. First, under the random split, several models sit near an F1 of 1.0 for long stretches, a level of performance that is difficult to explain as genuine fraud detection given how hard the task is under the honest split, and is much more consistent with the model exploiting structure that would not be available at real deployment time. Second, almost every model, under both splits, collapses toward zero F1 in the final stretch of the timeline, around time steps 43 to 49. This is not an artefact of our pipeline. Weber *et al*. [6] document a real event at time step 43, the shutdown of a dark market, after which the illicit class distribution changes abruptly and every method they tested struggled to keep up. Our curve reproduces that same collapse independently, which is a useful sanity check that our temporal split is behaving the way a genuine chronological evaluation should, and a reminder that even a perfectly leakage-free model still has to contend with real distribution shift, not only the leakage we set out to measure.

4.4. Where does the leakage live?

Going in, we expected the 71 neighbourhood-aggregate features to be the main leakage channel, since they are computed with knowledge of a node's surrounding graph structure. Figure 2 does not support that. All ten of the top ten features ranked by importance delta between the two splits are local transaction attributes, not neighbourhood aggregates, and the average importance delta for local features came out higher than for neighbourhood features in this run. That suggests even features that describe only the transaction itself still carry temporal signal, plausibly because local feature distributions drift over the dataset's 49 time steps, and a random split lets the model learn that drift instead of learning anything about actual fraud.

Figure 3 tests the structural explanation directly by progressively masking out GraphSAGE's edges. If the leakage were purely structural, stripping all the edges away should pull the random-split F1 down toward the temporal-split F1 and close the gap. Instead, the leakage delta stays roughly flat, moving from 0.35 with no edges removed to 0.38 with all of them removed. It does not shrink, and if anything it drifts slightly wider. That lines up with the feature-importance result above. A real share of the leakage in this dataset lives in the node features themselves, not only in how the graph is connected, which means edge-level defences like restricting neighbour sampling by time are necessary but not enough on their own to fully remove temporal-structural leakage here.

4.5. Failure modes: instability in EvolveGCN and MLP

Two of our models showed seed-dependent training collapse specifically under the harder temporal split, and this is worth treating as a finding in its own right rather than noise to average over. EvolveGCN's temporal-split F1 across the three seeds came out as 0.000, 0.110, and 0.262. One seed produced a model that predicted zero illicit nodes across the entire test set. We added gradient-norm clipping, capped at 1.0, specifically to stabilise EvolveGCN's recurrent weight-evolution step, and it did bring the standard deviation of the temporal-split F1 down from 0.183 before clipping to 0.132 after. It did not fix the seed-42 collapse though, which came out identical before and after clipping was added. Since gradient clipping only guards against exploding gradients, that points to the collapse more likely coming from vanishing gradients or an unlucky initialisation on the smaller, 34-snapshot temporal training window, rather than from exploding gradients. MLP had its own, separate collapse on one seed under the temporal split, with F1 of 0.009, precision of 1.0, and recall of 0.005. In effect, the model predicted almost no illicit nodes at all, and the handful it did predict happened to be correct. We have kept both instabilities visible in Table 1 and Table 2 rather than only reporting the mean, because the mean on its own hides how fragile some of these architectures become once the leaky shortcut is taken away.

4.6. A closer look: GNNExplainer

To go beyond the aggregate numbers, we ran GNNExplainer on the trained GraphSAGE model under the temporal split, picking one correctly-flagged illicit node and one missed illicit node from the same time step, shown in Figure 4. The result is the opposite of what we expected going in. The missed node's explanatory subgraph is by far the denser and more heavily clustered of the two, over a hundred neighbouring nodes across several tightly connected groups, while the correctly-flagged node's subgraph is a small, sparse tree of only a handful of nodes. If anything, this suggests density can work against GraphSAGE rather than for it here. A large, busy neighbourhood may dilute the illicit signal by averaging it together with many ordinary-looking transactions, while a small, distinctive local neighbourhood may be easier for the model to key on. That is a more troubling failure mode for a deployed system than the reverse would have been, since it implies the model is least reliable precisely on well-connected accounts, which are often the more consequential ones to catch.

5. Conclusion

Evaluating fraud-detection GNNs on the Elliptic dataset with a naive random split badly overstates how well these

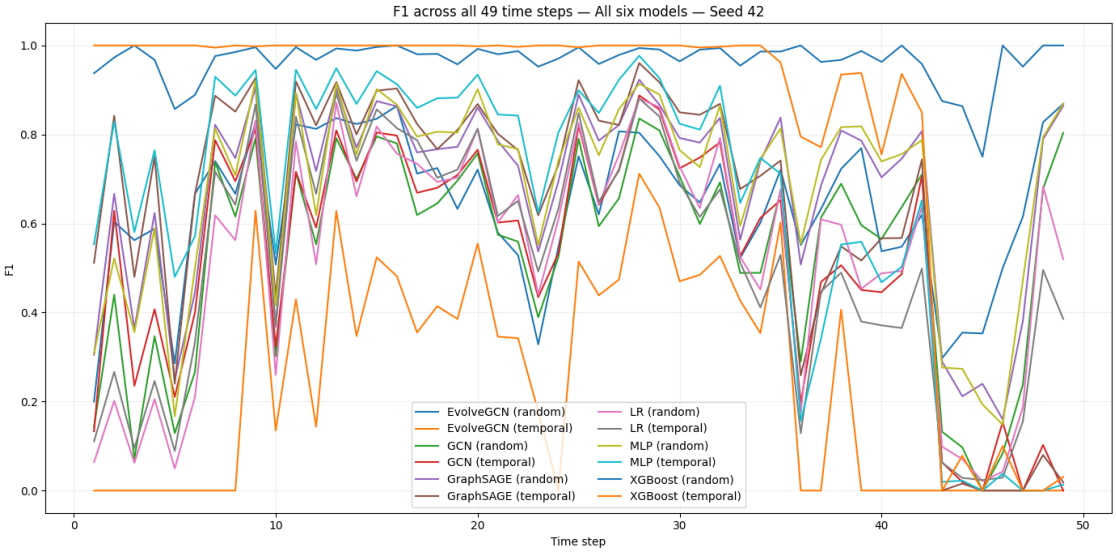


Figure 1. F1 on the illicit class at each of the 49 time steps, for all six models under both splits, seed 42.

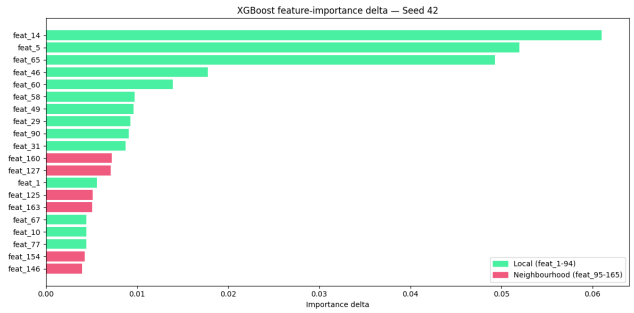


Figure 2. XGBoost feature-importance delta (random-split importance minus temporal-split importance) for the top 20 features, coloured by whether the feature is local (feat_1 to feat_94) or a neighbourhood aggregate (feat_95 to feat_165).

models would actually perform. Across six architectures, F1 on the illicit class fell by anywhere from 21.7 to 55.0 points once we switched to a strict chronological split with causal edge masking enforced. Because even our non-graph XGBoost baseline lost 29.0 points, and because removing every single edge from the graph only narrowed the gap slightly, we think temporal-structural leakage on this dataset comes substantially from feature-level drift over time, not only from message passing over future edges. That distinction matters for anyone building these systems in practice, since it means restricting a GNN's neighbour sampling by time helps but is not enough on its own to fully close the gap. We also came away with a genuine finding about training stability itself. Our custom EvolveGCN implementation stayed sensitive to random seed under the honest evaluation even after we added gradient-norm clipping, which tells us the harder condition affects not just the reported numbers but how reliably the model trains in the first place.

There are a few directions worth pursuing from here. One is running far more than three seeds to properly characterise how unstable EvolveGCN's training really is, rather than relying on three data points. Another is digging further into the feature-level leakage we found in Section 4.4, for example testing individual local features one at a time to isolate which specific attributes are carrying temporal drift. A third is extending the causal-masking protocol to a rolling, multi-window evaluation instead of a single train/validation/test cut, to check whether the leakage delta itself stays stable at different points across the dataset's 49-step history.

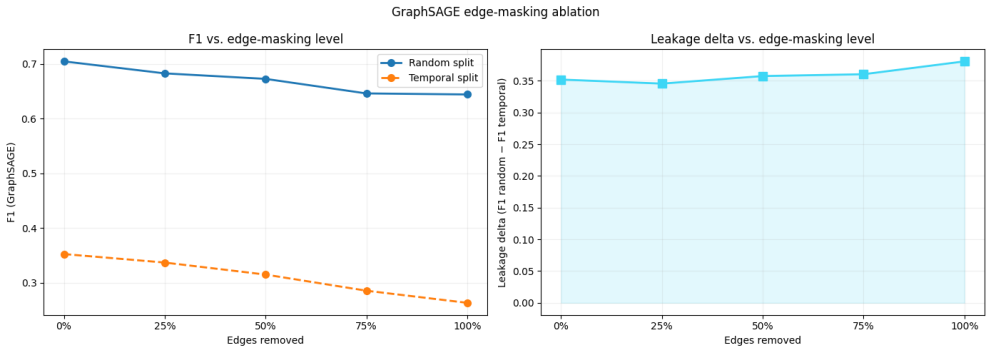


Figure 3. GraphSAGE F1 (left) and leakage delta (right) as edges are progressively removed from the graph, from 0% to 100%, under both splits.

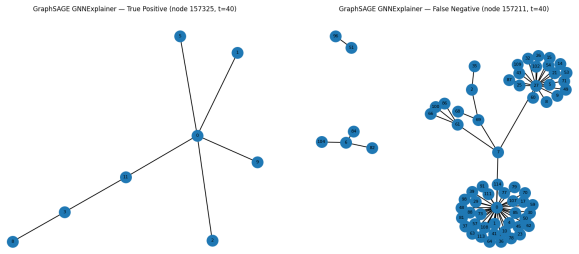


Figure 4. GNNExplainer subgraphs under the temporal split. Left: a true-positive prediction (node 157325, time step 40). Right: a false-negative prediction (node 157211, time step 40).

References

[1] W. L. Hamilton, R. Ying, and J. Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] S. Kaufman, S. Rosset, and C. Perlich. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 2012.

[3] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[4] S. Maganti. When graph structure becomes a liability: A critical re-evaluation of graph neural networks for bitcoin fraud detection under temporal distribution shift, 2026. arXiv:2604.19514.

[5] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. Schardl, and C. Leiserson. Evolvegcn: Evolving graph convolutional networks for dynamic graphs. In *AAAI Conference on Artificial Intelligence*, 2020.

[6] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. In *KDD Workshop on Anomaly Detection in Finance*, 2019.